VERSION 1.0

JANUARY 2025

DATA MINING

MODULE 4 - FEATURE SELECTION & DIMENSIONALITY REDUCTION

Author:

Yufis Azhar, Ir., S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY of MUHAMMADIYAH MALANG

TABLE OF CONTENTS

INTRODUCTION

PRACTICAL OBJECTIVES

1. Students are able to understand the basics of Python programming language
2. Students are able to understand Feature Selection and Dimensionality Reduction in Python programming language
3. Students are able to understand Data Splitting, Filter Methods, Wrapper Methods, Embedded Methods, PCA, LDA, and t-SNE

SOFTWARE / APPLICATION PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Hardware

- Laptop/Computer

3. Software (Choose one)

- Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook

4. Tutorial

- Using Google Colab : https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
- Using Google Colab Jupyter in VS Code : https://youtu.be/HOg13z1wI1A?si=RhEY2QY5bir-dzEH
- Using Jupyter Notebook : https://youtu.be/HQaCF2F_MRo?si=k6ubuGuCg-VS0tQe
- Install Anaconda : https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt

5. Google Colab Material

MODULE 4 DATA MINING - Feature Selection & Dimensionality Reduction.ipynb

KEYWORDS

Data Splitting, Feature Selection, Dimensionality Reduction

INTRODUCTION

In developing machine learning models, not all features in a dataset contribute equally to prediction accuracy. In fact, some features can be redundant or noisy, which actually reduces model performance.

Feature Selection and Dimensionality Reduction are two important approaches to address this issue in different ways:

- Feature Selection: Selecting the best subset of features from the existing features
- Dimensionality Reduction: Transforming features into new representations with lower dimensions

Why is it important?

1. Improve Model Accuracy: Removing irrelevant features reduces noise
2. Reduce Overfitting: Fewer features make the model more general
3. Speed Up Training: Computation is more efficient with fewer features
4. Data Visualization: Low-dimensional data is easier to visualize
5. Interpretability: Models with fewer features are easier to explain

Curse of Dimensionality

A phenomenon where model performance declines when the number of features is too large compared to the number of data samples. This occurs because:

- The feature space becomes very sparse (rare)
- The distance between data points becomes less meaningful
- The need for training data increases exponentially

A. Data Splitting

Before performing feature selection and dimensionality reduction, we must first understand the concept of data splitting. Data splitting is the process of dividing a dataset into several subsets for training, validating, and testing models.

Why is Data Splitting Important?

1. Preventing Data Leakage: Ensuring the model does not “see” test data during training.
2. Objective Evaluation: Measuring model performance on data it has never seen before.
3. Overfitting Detection: Comparing performance on train data vs. test data.
4. Hyperparameter Tuning: Using the validation set for parameter optimization.

Types of Data Split

1. Train-Test Split (70-30 or 80-20)

   The simplest division: some for training, some for testing.

```python
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

# Simple dataset example
np.random.seed(42)
X = np.random.rand(100, 5)  # 100 samples, 5 features
y = np.random.randint(0, 2, 100)  # Binary target

print(f"Total data: {len(X)}")

# Split 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% for test
    random_state=42,    # For reproducibility
    stratify=y          # Maintain class proportions
)

print(f"Training data: {len(X_train)} samples ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing data: {len(X_test)} samples ({len(X_test)/len(X)*100:.0f}%)")

# Check class proportions
print(f"\nClass proportions in training: {np.bincount(y_train)/len(y_train)}")
print(f"Class proportions in testing: {np.bincount(y_test)/len(y_test)}")
```

2. Train-Validation-Test Split (70-15-15 or 60-20-20)

   Split into three parts: training for model training, validation for hyperparameter tuning, and testing for final evaluation.

```python
# Method 1: Stepwise split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

print(f"Training data: {len(X_train)} samples ({len(X_train)/len(X)*100:.0f}%)")
print(f"Validation data: {len(X_val)} samples ({len(X_val)/len(X)*100:.0f}%)")
print(f"Test data: {len(X_test)} samples ({len(X_test)/len(X)*100:.0f}%)")
```

```python
# Method 2: Using numpy split (more flexible)
from sklearn.utils import shuffle

# Shuffle the data first
X_shuffled, y_shuffled = shuffle(X, y, random_state=42)

# Calculate split index
train_size = int(0.7 * len(X))
val_size = int(0.15 * len(X))

# Split
X_train = X_shuffled[:train_size]
y_train = y_shuffled[:train_size]

X_val = X_shuffled[train_size:train_size+val_size]
y_val = y_shuffled[train_size:train_size+val_size]

X_test = X_shuffled[train_size+val_size:]
y_test = y_shuffled[train_size+val_size:]

print(f"\nTrain: {X_train.shape}")
print(f"Validation: {X_val.shape}")
print(f"Test: {X_test.shape}")
```

```
Train: (70, 5)
Validation: (15, 5)
Test: (15, 5)
```

When to Use Each Split?

Data Splitting is used depending on the dataset to be tested. For details, see the table below:

Scenario	Split Recommendation	Reason
Small dataset (<1000 samples)	70-30 or 60-20-20 + Cross-validation	Maximize training data
Medium dataset (1000-10000)	70-15-15 or 80-10-10	Balance between training and validation
Large dataset (>10000)	80-10-10 or 90-5-5	Enough data for all subsets
Intensive hyperparameter tuning	Must use validation set	Avoid overfitting on test
Simple model, no tuning	Train-test only (80-20)	Validation not required

B. Feature Selection

What is Feature Selection? Imagine you want to predict whether a movie will be a box office hit or not. You have a lot of data: movie titles, director names, duration, poster colors, even the brand of shoes worn by the film crew.

Is all of that data important? Of course not. The brand of shoes worn by the crew most likely has nothing to do with the film's box office success. **Feature Selection** is the process of choosing the best “ingredients” (original features) that really affect the prediction results, and discarding “garbage” or irrelevant data.

1. Filter Methods

The Filter Method works like a sieve in the kitchen. Before ingredients are put into the “pot” (Machine Learning algorithm), we first filter them based on their statistical properties. This method is very fast because we don't need to try out models first.

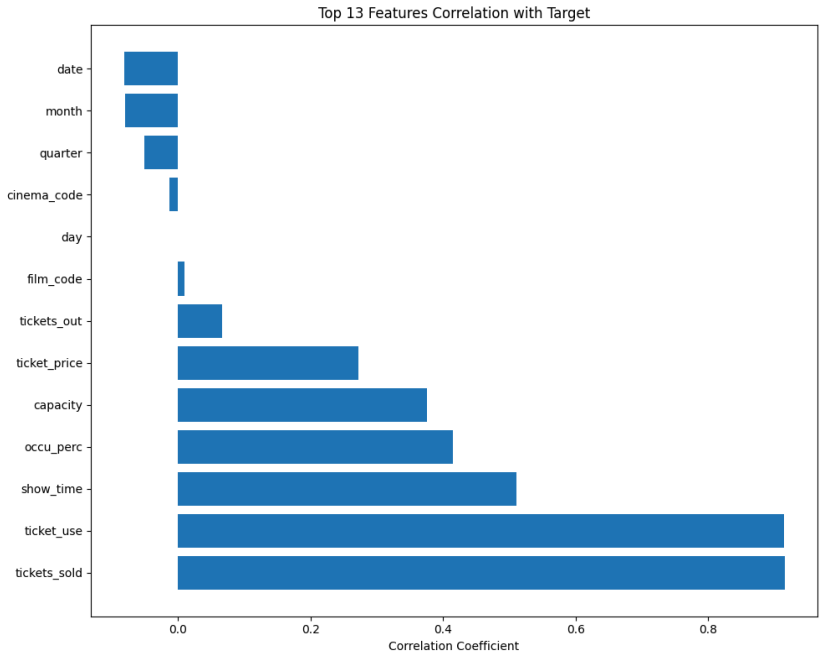
A. Correlation-Based Selection

This method measures how strong the relationship is between a feature (e.g., ticket price) and the target to be predicted (e.g., total sales).

Simple analogy: Imagine the relationship between cloudy skies and rain.

- If the clouds become darker, the chance of rain increases (this is called a strong positive correlation).
- If the temperature rises, ice cream sales may increase, but this has nothing to do with whether a movie will be a box office hit or not (this is called a weak/zero correlation).

Output :



Number of selected features (|correlation| > 0.3): 5
Selected features: ['tickets_sold', 'ticket_use', 'show_time', 'occu_perc', 'capacity']
Shape of dataset after selection: (142399, 6)

To determine which features are worth keeping, we look at their correlation values:

- Values close to 1 (Positive Correlation): “Proportional.” If the feature value increases, the target usually increases as well. Example: The greater the seating capacity, the higher the total sales.
- Values close to -1 (Negative Correlation): “Opposite”. If the feature value increases, the target actually decreases. Example: The more expensive the ticket price, the fewer the number of viewers.
- Values close to 0: “Irrelevant”. This feature has no clear effect on the target. This is the feature we should discard.

Important: We usually set a threshold. For example, we only take features that have a correlation value above 0.3 or below -0.3. Features in between are considered “noise” that can slow down the computer and reduce the accuracy of the model.

B. Chi-Square Test

What is the Chi-Square Test? If correlation is used to examine the relationship between numbers (such as ticket prices and income), Chi-Square is used to examine the relationship between group or category data.

This test seeks to determine: “Does this feature have a strong relationship with the target, or is the relationship merely coincidental?”

A simple analogy: “Movie Genres and Types of Snacks” Imagine you want to know if there is a relationship between the movie genres people watch and the types of snacks they buy.

- If horror movie viewers always buy salty popcorn, while romantic movie viewers always buy chocolate, then there is a strong relationship between genre and snack (high Chi-Square score).
- However, if horror, romantic, and action movie viewers all buy snacks randomly (no pattern), then there is no relationship (low Chi-Square score).

Output :

```
Fitur terpilih dengan Chi-Square Test (top 10):
['film_code', 'cinema_code', 'tickets_sold', 'ti

Chi-Square Scores:
         Feature     Chi2_Score
5      occu_perc  11183.736080
2   tickets_sold   8914.656105
8       capacity   4038.308981
4      show_time   3435.968854
7     ticket_use   3364.344313
3    tickets_out   3060.959750
1    cinema_code   3024.662755
0      film_code   2541.470580
9           date   2329.865409
12           day   1725.053221
11       quarter   1723.346981
10         month   1703.146220
6   ticket_price   1001.651413
```

Explanation:

Why use MinMaxScaler? Chi-Square is like a scale that cannot accept negative values. Therefore, we must ensure that all our data is 0 or higher (non-negative) before testing.

What does the Chi2 Score mean?

- The higher the score: This means that the feature has a strong "inner connection" with the target. This feature is very important to keep because it can help the model predict more accurately.
- The lower the score: This means that the feature is indifferent to the target. Removing this feature will usually not hurt your model's performance.

When to Use This? Use Chi-Square if you have a lot of categorical data (such as: Member Type, Movie Theater Location, Screening Day) and want to know which ones most determine the final outcome (such as: Popular or Not).

C. Variance Threshold

What is Variance Threshold? In the world of data, **"variance"** is a measure of how diverse the contents of a column are. If a column contains almost identical contents (for example, 99% of rows contain the same number), then that feature is considered to provide no information for the machine to learn.

A Simple Analogy: "Movie Audience Survey" Imagine you are conducting a survey of 1,000 movie audience members to predict whether they will buy popcorn. You ask two questions:

- Are you human? (99% answer: Yes) -> Low Variance. This answer is useless because everyone gives the same answer. You cannot distinguish popcorn buyers from this question alone.
- How old are you? (Answers vary: 15, 25, 40, 60) -> High variance. This data is useful because age differences may determine who likes to buy popcorn and who doesn't.

Variance Threshold serves to discard the first type of “boring” questions because their content is uniform.

Output :

```
Variance setiap fitur:
ticket_price    1.104723e+09
capacity        9.084341e+05
tickets_sold    7.823722e+04
ticket_use      7.812869e+04
cinema_code     2.548841e+04
date            4.217073e+03
film_code       1.309294e+03
occu_perc       5.131786e+02
day             8.007812e+01
show_time       9.344348e+00
tickets_out     8.551126e+00
month           4.818245e+00
quarter         6.551479e-01
dtype: float64

Jumlah fitur dengan variance > 0.1: 13
Fitur terpilih: ['film_code', 'cinema_code',
```

Explanation :

Why Are Certain Features Missing? If certain features are missing after running the algorithm, it means that those features are too uniform. The machine considers those features to be a computational “burden” without providing meaningful clues for prediction.

Determining the Threshold Value:

- If Threshold = 0: We only discard features whose values are exactly the same across all rows.
- The higher the threshold value you provide, the more “aggressive” the filtering will be. Only features with high data diversity will be allowed to pass.

2. Wrapper Methods

What are Wrapper Methods? If Filter Methods work like a sieve (looking only at data properties), Wrapper Methods work like an “audition.” We actually train the Machine Learning model and see how well it works with certain feature combinations.

This is much more accurate because features are selected based on actual performance, but it takes longer because the computer has to “learn” repeatedly.

A Simple Analogy: “Selecting a National Soccer Team”

- Filter Method: You select players based solely on height or running speed (statistics).
- Wrapper Method: You have the players compete directly on the field. You see who plays well with the team, then select the best based on the results of those matches.

A. Recursive Feature Elimination (RFE)

RFE is an elimination system. It starts with all features, then discards the weakest ones one by one until the “Dream Team” (the best features) we want remains.

RFE analogy: “Survival Show (Example: MasterChef)”

1. In the first round, all participants (features) cook together.
2. The judges (ML Model) evaluate their performance.
3. The worst contestant is eliminated.
4. The next round begins with the remaining contestants, then another is eliminated.
5. This process is repeated until the top 10 best contestants remain.

```
# ==========================
# 2. WRAPPER METHODS
# ==========================

## A. Recursive Feature Elimination (RFE)

from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import pandas as pd # Ensure pandas is imported for qcut

num_bins = 5 # You can adjust the number of bins as needed
y_binned = pd.qcut(y, q=num_bins, labels=False, duplicates='drop')

# Split data using the binned target for stratification
X_train, X_test, y_train_binned, y_test_binned = train_test_split(
    X, y_binned, test_size=0.3, random_state=42, stratify=y_binned
)

# Create model
model = LogisticRegression(max_iter=10000, random_state=42)

# Apply RFE to select 10 best features
n_features_to_select = min(10, X.shape[1])
rfe_selector = RFE(estimator=model, n_features_to_select=n_features_to_select, step=1)
rfe_selector.fit(X_train, y_train_binned) # Use binned target for fitting RFE

# Selected features
selected_features_rfe = X.columns[rfe_selector.support_].tolist()

print(f"\nFeatures selected with RFE (top {n_features_to_select}):")
print(selected_features_rfe)

# Feature ranking (1 = best)
feature_ranking = pd.DataFrame({
    'Feature': X.columns,
    'Ranking': rfe_selector.ranking_
}).sort_values('Ranking')

print("\nFeature Ranking:")
print(feature_ranking)
```

```
# Compare accuracy before and after RFE
# Model with all features
model_all = LogisticRegression(max_iter=10000, random_state=42)
model_all.fit(X_train, y_train_binned)
y_pred_all = model_all.predict (X_test)
acc_all = accuracy_score(y_test_binned, y_pred_all)

# Model with RFE features
X_train_rfe = rfe_selector.transform(X_train)
X_test_rfe = rfe_selector.transform(X_test)

model_rfe = LogisticRegression(max_iter=10000, random_state=42)
model_rfe.fit(X_train_rfe, y_train_binned)
y_pred_rfe = model_rfe.predict(X_test_rfe)
acc_rfe = accuracy_score(y_test_binned, y_pred_rfe)

print(f"\nAccuracy with all features ({X_train.shape[1]} features): {acc_all:.4f}")
print(f"Accuracy with RFE ({n_features_to_select} features): {acc_rfe:.4f}")
print(f"Difference: {acc_rfe - acc_all:.4f}")
```

Explanation :

- Ranking 1: This means that the feature is the “core” of your data. It is never eliminated during the audition process.

- High Ranking (2, 3, etc.): This means that the feature is eliminated early because it is considered to be of little help to the model in making predictions.

Why Does Accuracy Increase? Often, too many features (noise) confuse the model. By removing unimportant features through RFE, our model becomes more “focused” and can provide more accurate predictions, or at least equally good predictions, but with less work.

B. Forward Selection

What is Forward Selection? If RFE (Recursive Feature Elimination) works with an elimination system (starting with many and then reducing), Forward Selection works the opposite way. We start with an empty hand (0 features), then try each feature one by one and only include those that most improve the model's performance.

A Simple Analogy: “Building a Music Band”

- RFE: You call 50 musicians to the stage, then send the worst performers home one by one until you are left with the best band.
- Forward Selection: The stage is empty. You try one guitarist, then one drummer, then one vocalist. You only add personnel if the band sounds significantly better than before. If adding a pianist makes the sound chaotic, then the pianist is not invited.

Output :

```
Fitur terpilih dengan Forward Selection (top 10):
['cinema_code', 'tickets_sold', 'tickets_out', 'ticket_price',
 'ticket_use', 'capacity', 'date', 'month', 'quarter', 'day']
```

Explanation:

Why Use Random Forest? In feature selection, we often use different models to see other perspectives. Random Forest is excellent for seeing which features have high “predictive power” (feature importance) in a non-linear way.

Why is the process slower? Because the computer must try:

1. Try Feature A -> Check Accuracy.
2. Try Feature B -> Check Accuracy.

3. Try (A + B) -> Check Accuracy.

... and so on until the best combination of 10 features is found.

When to Use Forward Selection? This method is ideal if you have thousands of features but suspect that only a few (e.g., just 5 or 10) are truly influential. Starting from scratch is far more efficient than eliminating one by one from thousands of features.

C. Backward Elimination

What is Backward Elimination? If Forward Selection starts from zero and continues to add features, Backward Elimination is the opposite. We start by putting all the features into the model, then one by one the most useless features or those that worsen the model's performance will be discarded.

Simple analogy: “Carving a statue from stone”

- Forward Selection: Like building a house from bricks. You start with an empty lot, then add bricks one by one until the house is complete.
- Backward Elimination: Like a sculptor creating a statue. You start with a large block of stone (all features), then remove the unnecessary parts until a beautiful statue (the best features) is formed.

Output :

```
Backward Elimination (top 10):

['film_code', 'cinema_code', 'tickets_sold', 'tickets_out',
'ticket_price', 'ticket_use', 'date', 'month', 'quarter', 'day']
```

Explanation:

Why is it different from RFE? You may ask, **“What is the difference between this and RFE?”**. Although similar, RFE discards features based on the model's internal weights (such as coefficient values), while Backward Elimination discards features by trial and error (removing one feature, checking the accuracy, and if the accuracy remains good, the feature can be discarded).

When to Use This? This method is excellent if you suspect that your features work together (have strong interactions). By including everything at the beginning, the model can see the big picture first before starting to remove the smallest ones.

Time Note: This method is usually the slowest compared to others. Imagine the computer having to run the model multiple times with a very large number of features at the beginning of the process.

3. Embedded Methods

What are Embedded Methods? If Filters work like sieves and Wrappers work like auditions, then Embedded Methods work like “Natural Selection”. Unuseful features will naturally fall away as the model learns. This is the best of both worlds: as accurate as Wrappers but as fast as Filters.

A. Lasso Regularization (L1)

Lasso is a type of mathematical model that has a unique property: it is very stingy. It imposes a “penalty” or tax on features that do not make a real contribution. If a feature is considered unimportant, Lasso will force its importance value (coefficient) to zero.

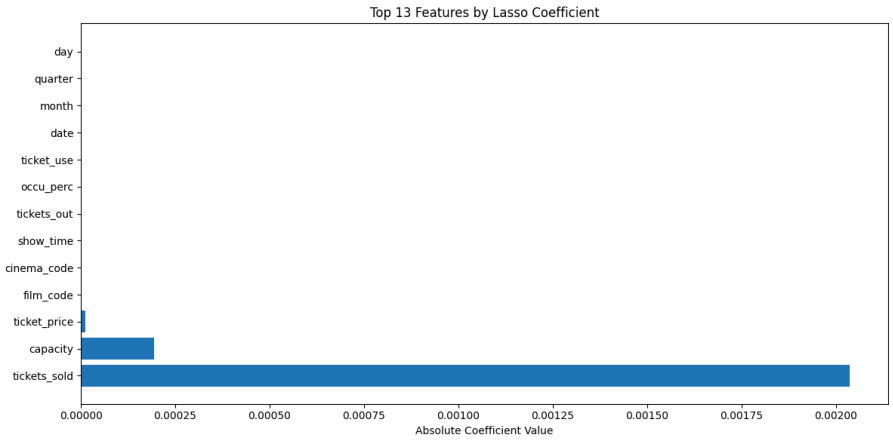
A Simple Analogy: “Packing Your Suitcase for Vacation” Imagine you want to go on vacation, but your suitcase is very small and there is a rule: “Every item you bring has a weight tax.”

- Clothes and Passport: Very important. Even though there is a weight tax, you still bring them because they are very useful.
- Spare Key Chain: Not very important. Because of the weight tax, you decide to leave it at home (its value becomes zero).

Lasso works like this weight tax rule; it forces you to only bring items that are truly essential.

Output :

Number of selected features: 3
Selected features: ['tickets_sold', 'capacity', 'ticket_price']



Explanation:

1. **Why Are Some Features Missing?** If you see features that do not appear in the graph or list, it is because Lasso has assigned a value of 0 to those features. This means that, according to Lasso's mathematics, those features are “junk” that only add to the model's load.
2. **Key Advantage:** Lasso is great because it does two jobs at once: it builds a model and selects features. You don't have to bother with separate feature selection.
3. **When to Use It:** Lasso is very effective if you have many features that are interrelated (multicollinearity). It will tend to select the best one and reject the others.

B. Ridge Regularization (L2)

What is Ridge Regularization? If Lasso (L1) is “harsh” by directly discarding unimportant features (making them zero), Ridge (L2) is more “wise.” Ridge does not discard features at all, but rather minimizes the influence (coefficient) of features that are considered less important so as not to damage the prediction.

Simple analogy: “Dimming the Lights (Dimmer Switch)” Imagine you are on a theater stage with 50 spotlights (features).

- Lasso (L1): Will immediately turn off (switch OFF) the dim lights or those not focused on the actors.
- Ridge (L2): Does not turn off any lights, but it will dim the unimportant lights and only allow the main lights to remain bright. All lights remain on, but their influence varies.

Output :

```
Top 15 fitur berdasarkan Ridge Coefficient:
         Feature  Coefficient
10         month     1.590973
11       quarter     0.316840
4      show_time     0.062857
12           day     0.047616
9           date     0.045576
5      occu_perc     0.035593
0      film_code     0.004325
1    cinema_code     0.000613
8       capacity     0.000388
3    tickets_out     0.000345
2   tickets_sold     0.000200
7     ticket_use     0.000145
6   ticket_price     0.000011
```

Top 10 selected features: ['month', 'quarter', 'show_time', 'day', 'date', 'occu_perc', 'film_code', 'cinema_code', 'capacity', 'tickets_out']

Explanation:

1. **Why are there no zero coefficients?** This is the inherent nature of Ridge. It believes that every piece of information (feature) may have a small contribution, so it does not discard it. It only suppresses the values of "noisy" features so that their values are very small, close to zero, but never actually zero.
2. **When to Use Ridge?** Use Ridge if you feel that all features in your dataset play an important role and you don't want to lose any information, but you want your model to be less sensitive to unstable data.
3. **Comparison of Lasso vs. Ridge:**
    - Lasso: Good for simplifying the model (discarding features).
    - Ridge: Good for handling closely related data (multicollinearity) without having to discard features.

C. Tree-Based Feature Importance

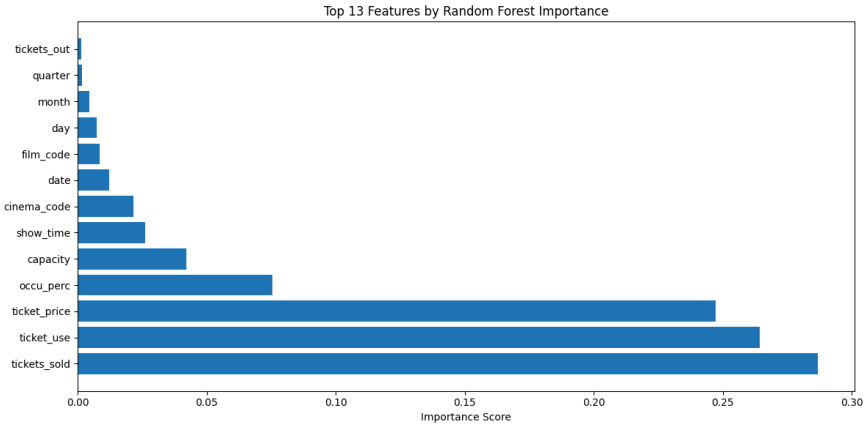
What is Tree-Based Importance? Models such as Random Forest work by creating hundreds of "Decision Trees." Each tree will ask the data: "Is the ticket price expensive?" or "Is it a holiday?"

The features that are most often used to make accurate decisions will receive a high Importance score.

A Simple Analogy: "Asking 100 Strategy Experts" Imagine you want to know the most decisive factor in determining whether a movie will be a box office hit. You ask 100 cinema experts (Random Forest).

- Each expert will note down the factors they use to make their guess (e.g., Actor Name, Genre, or Duration).
- If almost all experts consistently ask "Who is the lead actor?" to give the correct answer, then the "Actor" feature is considered to have very high Importance.
- If the "Poster Color" feature is rarely asked or does not help them guess correctly, then its importance score will be very low.

Output :



Explanation:

1. **Why Use This?** Unlike correlation, which only looks at linear relationships, Random Forest can see complex relationships. For example, ticket prices may not be important on their own, but they become very important when combined with the location of the cinema.
2. **How to Read the Scores:** All importance scores add up to 1.0 (100%). So, if a feature has a score of 0.2, it means that feature contributes 20% to the model's decision-making power.
3. **Threshold:** We usually discard features with very small scores (e.g., below 0.01) because they are considered “spectators” that do not contribute significantly to the prediction.

4. Comparison of Feature Selection Methods

The following is a comparison between several Feature Selection Methods:

Method	Evaluation Criteria	Advantages	Disadvantages
Filter	Correlation/independent statistics	Fast, easy to apply	Does not consider feature interactions
Wrapper	Evaluation with model algorithms	Considers feature interactions	Computationally expensive
Embedded	Selection during model training	Integrated and efficient	Dependent on the model

C. Dimensionality Reduction

What is the difference with Feature Selection? If Feature Selection is like choosing the 5 best spices from the spice rack, then Dimensionality Reduction is like putting all the spices in a blender and making a spoonful of secret spice paste. It contains the essence of all the original spices, but its form has changed and is much more concise.

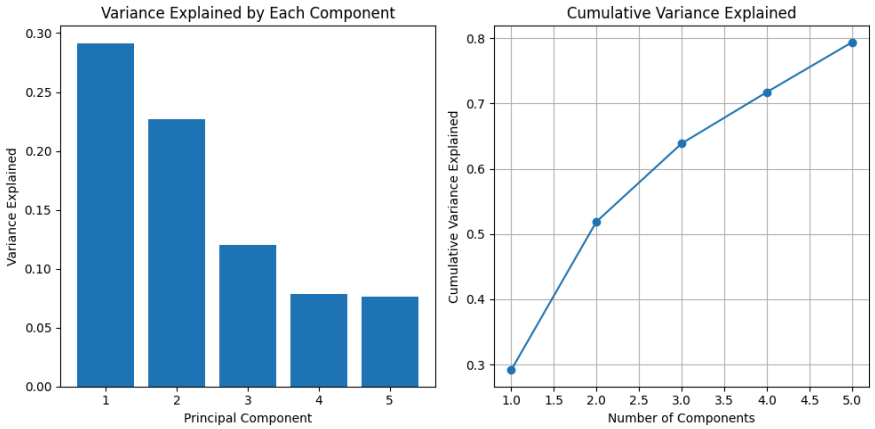
1. Principal Component Analysis (PCA)

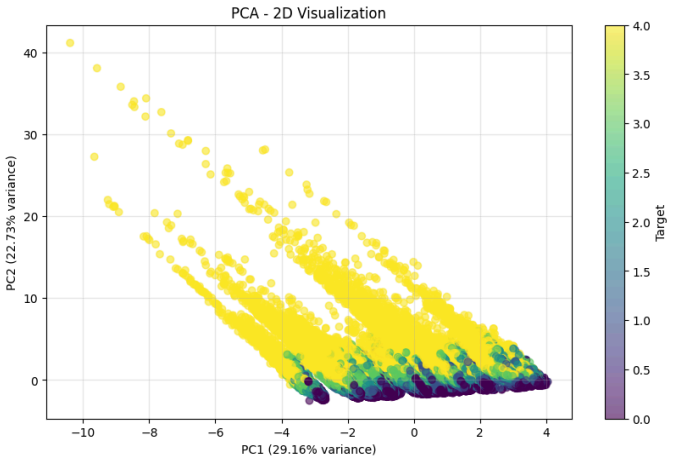
PCA is a technique for summarizing data that has many columns into just a few new columns (called Principal Components) without losing too much important information.

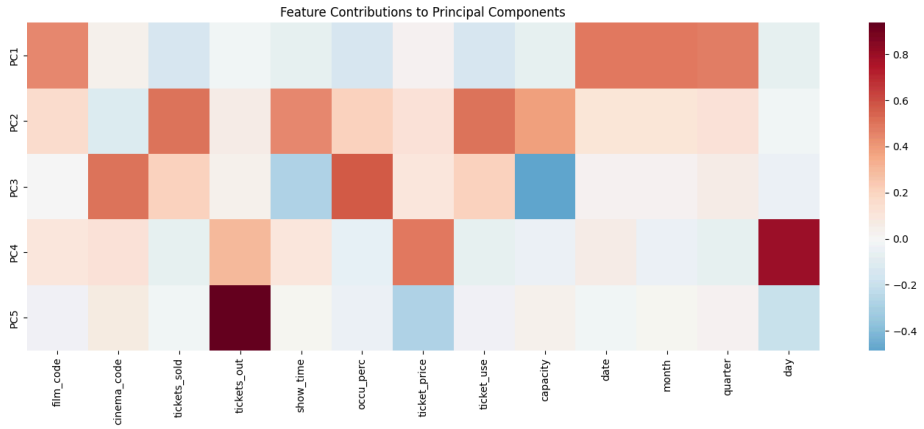
Simple analogy: “Photographing a 3D object” Imagine you have a statue (3D data: length, width, height). You want to describe the shape of the statue to a friend through a photo (2D).

- You will look for the best photo angle (Principal Component) where the entire shape of the sculpture is most clearly visible and not much of it is covered.
- The photo has lost the “depth” dimension, but because you took the right angle, your friend still knows what the sculpture is. That is PCA: projecting data into a lower dimension from the most informative perspective.

Output :







Explanation:

- Principal Components (PC1, PC2, etc.): These are new features created by PCA. PC1 always carries the most information (variance), PC2 the second most, and so on.
- Cumulative Variance: See the orange line graph. If you can capture > 90% of the information with just 2 components, then you no longer need to use dozens of original columns. Just use these two new columns for your model.
- Contribution Heatmap: This section shows the "recipe" for the pasta sauce. For example, PC1 may contain 80% of the ticket_price feature and 20% of the seat_capacity feature. This helps us understand the meaning of the new columns.

Note: PCA is an unsupervised method. This means that when summarizing data, it does not look at the target (y). It only focuses on finding the most variable features. If you want to summarize data while looking at the target, we will use LDA in the next section.

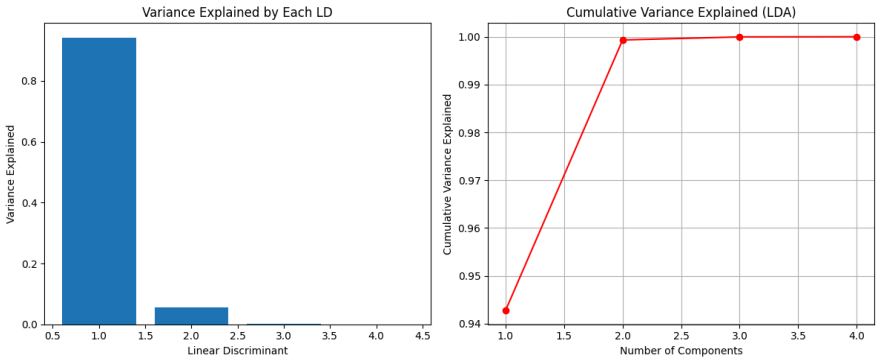
2. Linear Discriminant Analysis (LDA)

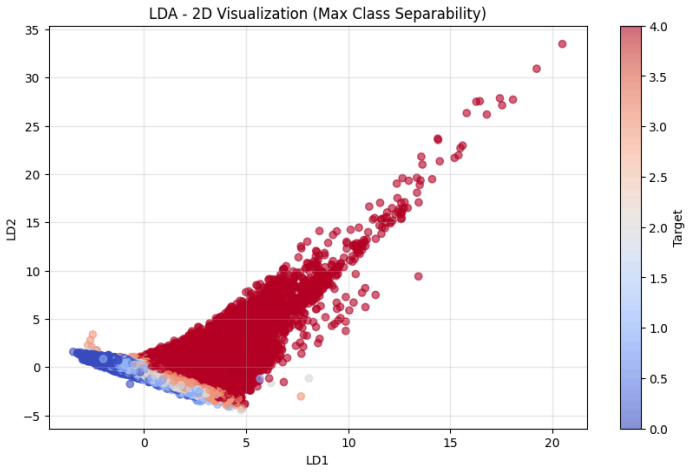
What is LDA? If PCA looks for the “photo angle” that captures the most detail (variance), LDA looks for the “photo angle” that can best distinguish one group from another. LDA is a supervised method, meaning it must “see” the target label (y) in order to know how to neatly separate the data.

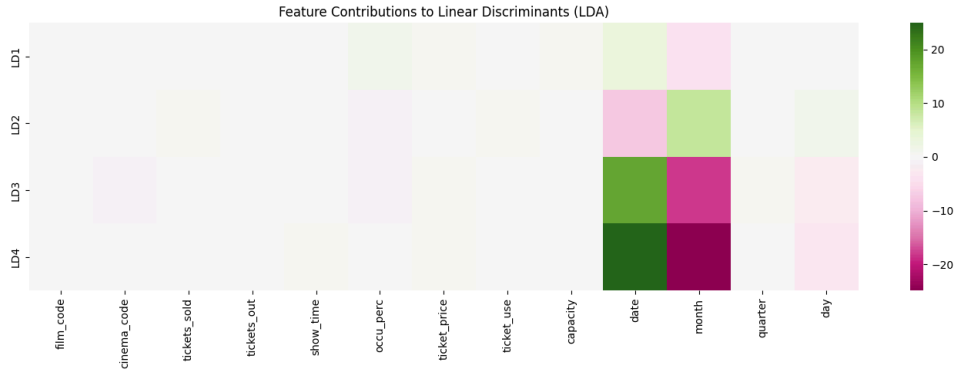
Simple analogy: “Separating a mixture of candies” Imagine that in front of you there is a mixture of red and blue marbles scattered on a table.

- PCA: Finds a way to arrange the marbles so that they are spread as far apart as possible and cover as much of the table as possible (focuses on dispersion/variance).
- LDA: Finding a boundary line on the table so that all the red marbles are grouped on the left and the blue marbles are grouped on the right. LDA focuses on maximizing the distance between groups and minimizing the distance within groups.

Output :







Explanation:

1. **Class Separation:** Look at the LDA visualization results. If the color groups (categories) appear to be separated far apart from each other (not overlapping like PCA), it means that your original features do indeed have strong distinguishing patterns.
2. **Why is LDA Limited?** Mathematically, to separate n groups, you only need a maximum of n−1 dividing lines. That is why the number of LDA components is much smaller than the original features.
3. **When to Use LDA?** Use LDA if your main goal is to build a classification model. LDA will prepare new features that have been "cleaned up" so that your classification model can more easily recognize the differences between classes.

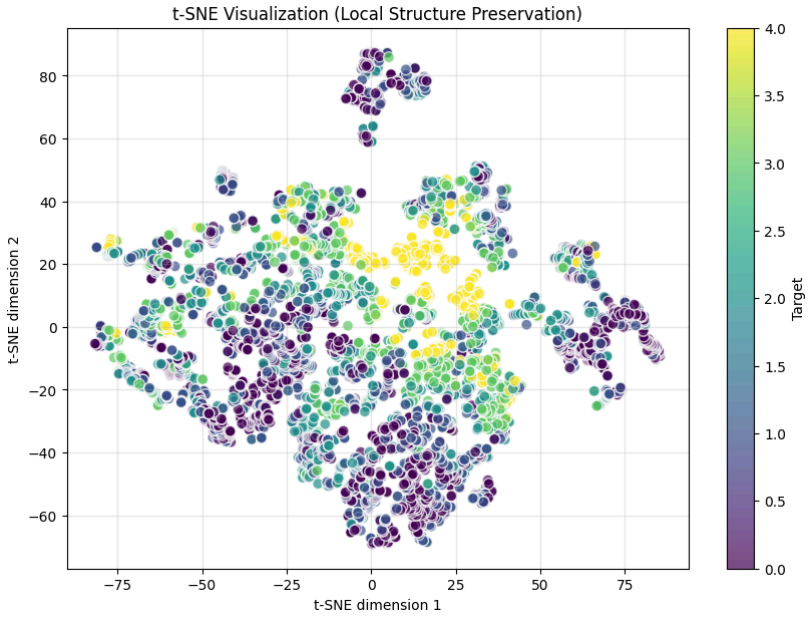
3. t-Distributed Stochastic Neighbor Embedding (t-SNE)

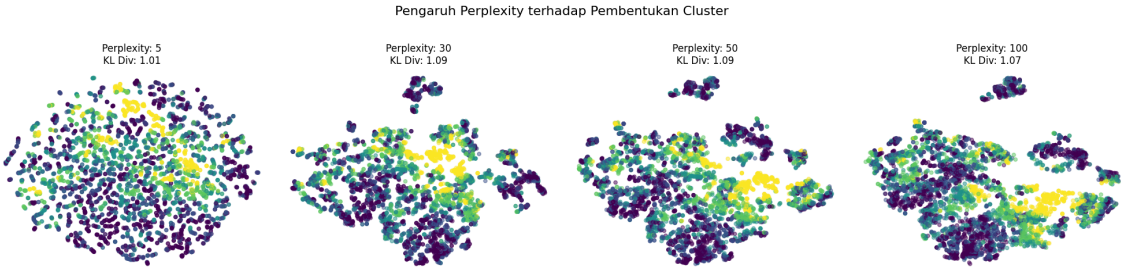
What is t-SNE? Unlike PCA or LDA, which are linear (like viewing data from a straight angle), t-SNE is a non-linear method. Its main task is visualization. t-SNE works hard to ensure that data that are originally neighbors or similar in high dimensions remain neighbors when transferred to a 2D map.

A simple analogy: "Peeling an orange" Imagine you have an orange whose skin is drawn on a map of the world (the earth is round/3D). You want to transfer the map onto flat paper (2D).

- If you force it linearly, the map will tear or the shape of the continents will become distorted.
- t-SNE works like carefully peeling the orange skin, then stretching and sticking it piece by piece onto the paper so that the continents that were originally neighbors remain close together.

Output :





Explanation :

1. **Cluster:** If you see separate clusters on the graph, it means that t-SNE has successfully found groups of data that have similar properties. The closer the distance between two points, the more similar the two data points are.
2. **KL Divergence:** You will see this value in the code output. Think of it as an "Error Score." The lower the value, the more successfully t-SNE has moved the data to a 2D map without damaging the relationships between neighbors.
3. **Perplexity:** This is the most important parameter in t-SNE.
    - Low Perplexity (e.g., 5): t-SNE only cares about very close neighbors (the result will look like small scattered dots).
    - High Perplexity (e.g., 50): t-SNE looks at the big picture (the result will form larger, more cohesive clusters).

4. Comparison of PCA vs LDA vs t-SNE

The following is a comparison of the three methods of Dimensionality Reduction:

Aspect	PCA	LDA	t-SNE
Type	Unsupervised	Supervised	Unsupervised
Linear	Yes	Yes	No
Purpose	Maximize variance	Maximize class separation	Preserve local structure
Speed	Fast	Fast	Slow
Interpretability	Moderate	High	Low
New data transformation	Yes	Yes	No
Use case	Compression, noise reduction	Classification	Visualization

CODELAB (20%)

In this exercise, you will perform feature selection and dimensionality reduction on the **Super Market Sales** dataset.

Dataset: [Super Market Sales]

Alternative with Kaggle

```
import kagglehub
import shutil
import os

# 1. Download dataset
path = kagglehub.dataset_download("akashbommidi/super-market-sales")

# 2. Specify the destination folder name in /content
destination = "/content/super-market-sales"

# 3. Move or copy the folder
if not os.path.exists(destination):
```

```
    shutil.copytree(path, destination)
    print(f"The dataset has been successfully copied to: {destination}")
else:
    print("The folder is already in /content.")
```

Instruction :

1. **Perform Exploratory Data Analysis (EDA) (15 points)**
    - Display descriptive statistics
    - Check for missing values
    - Visualize the distribution of the target variable (quality)
    - Create a correlation heatmap
2. **Perform Feature Selection (30 Points)**
    - Use correlation-based selection and use RFE
    - Then compare the results with visualization
3. **Next, perform Dimensionality Reduction (30 Points)**
    - Apply PCA and determine the number of components
    - Then create a visualization
4. **Show the Evaluation Model to the Assistant by showing: (25 Points)**
    - Comparison of Accuracy between Feature Selection and PCA that you have tried
    - Create a visualization to make it easier to explain to the Assistant

LAB ASSIGNMENTS (80%)

ODD NIM

Dataset: [Customer Churn] Link: [Dataset for ODD NIM]

Alternative with Kaggle

```
import kagglehub

# Download latest version

path = kagglehub.dataset_download("sohailaelsayed/customer-churn")

print("Path to dataset files:", path)
```

Instructions:

1. **Feature Selection (25%)**
    - Perform a complete EDA that displays the entire dataset and also provide visualizations. (20 points)
    - Apply two different feature selection methods, namely the Chi-Square Test and Forward Selection. (50 points)
    - Create a comparison visualization and comparison table between the two Feature Selection methods. (30 Points)
2. **Dimensionality Reduction (25%)**
    - Apply PCA: (40 Points)
    - Apply LDA: (40 Points)
    - Compare PCA with LDA using visualizations and comparison tables of the best methods (20 points)
3. **Model Comparison (20%)**
    - **Create at least 5 pipelines with: (45 points)**
        - Baseline (all features)
        - Best feature selection method
        - PCA
        - LDA
        - Combination (feature selection + PCA)
    - **Then evaluate using: (45 Points)**
        - Accuracy
        - Precision
        - Recall
        - F1-Score
        - Confusion Matrix
    - **Visualize the comparison in tables and graphs (10 Points)**
4. **Explain to the Assistant (30%)**

EVEN NIM

Dataset: [Financial Transaction Fraud Detection] Link: Dataset for EVEN NIM

Alternative with Kaggle

```
import kagglehub
# Download latest version
path =
  kagglehub.dataset_download("bertnardomariouskono/financial-transaction-f
  raud-detection")
print("Path to dataset files:", path)
```

Instructions:

1. **Feature Selection (15%)**
    - Perform a complete EDA that displays the entire dataset and also provide visualizations. (20 points)
    - Apply two feature selection methods, namely Correlation-Based and Backward Elimination. (50 points)
    - Create a comparison visualization and comparison table between the two Feature Selection methods. (30 Points)
2. **Dimensionality Reduction (15%)**
    - Apply PCA (40 Points)
    - Apply t-SNE (40 Points)
    - Compare PCA with t-SNE with visualizations and comparison tables of the best methods (20 points)
3. **Model Comparison & Analysis (20%)**
    - **Create at least 5 pipelines with: (45 points)**
        - Baseline (all features)
        - Best feature selection method
        - PCA
        - LDA
        - Combination (feature selection + PCA)
    - **Then evaluate using: (45 Points)**
        - Accuracy
        - Precision
        - Recall
        - F1-Score
        - Confusion Matrix
    - **Visualize the comparison in tables and graphs (10 Points)**
4. **Explain to the Assistant (30%)**

GRADING CRITERIA & DETAILS

Activity	Description	Weight
Codelab		**20%**
Lab Assignments	Feature Selection (Odd: Chi-Square/Forward, Even: Corr/Backward)	15%
	Dimensionality Reduction (Odd: PCA/LDA, Even: PCA/t-SNE)	15%
	Model Comparison & Pipeline Analysis	20%
	Explanation to Assistant	30%
Sub Total		**80%**
Total		**100%**